

CAR SERVICE & MAINTENANCE REMINDER APP

AUTOMATED TESTING USING APPIUM & JAVA

CAPSTONE PROJECT DOCUMENTATION

Application Used:

Drivvo – Car Maintenance Reminder App

Candidate Id & Name:

57152 – Abhishek Suman



Scan to view live project profile

TABLE OF CONTENTS

1. Abstract
2. Introduction
3. Problem Statement
4. Real World Scenario
5. Objectives of the Project
6. Terminologies
7. Application Overview – Drivvo
8. Application Architecture
9. Automation Framework Architecture
- 10.Environment Setup
- 11.Test Case Design
- 12.Detailed Test Cases
- 13.Automation Implementation Using Appium and Java
- 14.Automating Add Reminder Functionality
- 15.Automating Edit Reminder Functionality



16. Automating Delete Reminder Functionality

17. Screenshot Capture on Test Failure

18. Test Execution Process

19. Test Execution Report

20. Challenges Faced During Implementation

21. Solutions Implemented

22. Industry Relevance of the Automation Framework

23. Deliverables

24. Learning Outcomes

25. Future Enhancements

26. Conclusion

27. References

ABSTRACT

This project presents a comprehensive implementation of mobile automation testing for a Car Service & Maintenance Reminder application using Appium and Java. The objective of this project is to validate the reliability and correctness of reminder management functionalities in a real-world vehicle maintenance application.

Vehicle maintenance plays a critical role in ensuring safety, performance, and compliance with legal regulations. Car owners are required to perform periodic servicing, oil changes, insurance renewals, and pollution control checks. However, due to busy schedules and lack of tracking systems, these important tasks are often forgotten.

To solve this issue, mobile applications such as **Drivvo** provide reminder management systems. However, the effectiveness of such applications depends entirely on the accuracy of reminder functionality.

This project focuses on automating the core reminder operations, including:

- Adding a service reminder (Oil Change)
- Editing an existing reminder
- Deleting a reminder
- Capturing screenshots on failure
- Generating automated HTML reports

The automation framework was developed using Java and Appium with UiAutomator2 driver for Android devices. The project demonstrates practical mobile automation testing concepts aligned with industry standards.

INTRODUCTION

With the rapid growth of smartphone usage, mobile applications have become an essential part of daily life. From banking to healthcare and vehicle management, users rely on mobile apps for convenience and efficiency.

One important category of mobile applications is vehicle maintenance reminder systems. Car owners must regularly monitor:

- Periodic servicing
- Engine oil changes
- Insurance renewal dates
- Pollution Under Control (PUC) certificate validity

Failure to maintain vehicles properly can result in:

- Reduced vehicle performance
- Increased maintenance cost
- Safety risks
- Legal penalties

Mobile applications help solve this problem by allowing users to create and manage reminders.

However, if reminder functionalities fail — such as not saving correctly, not updating properly, or not deleting — the application becomes unreliable. Therefore, automation testing becomes necessary.

This project implements mobile automation testing using Appium and Java to ensure that reminder management features operate correctly.

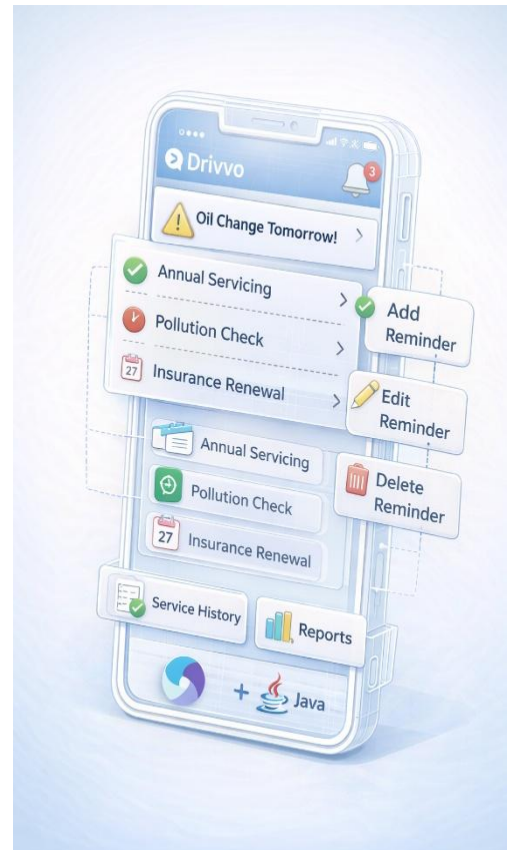
PROBLEM STATEMENT

Vehicle maintenance is a recurring and essential responsibility for car owners. While reminder applications assist users in tracking important maintenance activities, their reliability is crucial.

The problem addressed in this project is:

How can a mobile-based Car Service & Maintenance Reminder application be reliably tested using Appium and Java to ensure reminder creation, update, and deletion functionalities work correctly across Android devices?

To solve this, an automated testing solution is developed that validates core reminder operations.



REAL WORLD SCENARIO

Consider a working professional who owns a car. Due to daily responsibilities, they forget to change engine oil after reaching 5000 km.

They install **Drivvo** to manage reminders.

They:

- Add Oil Change reminder
- Later modify it
- Finally delete it after service completion

If any of these steps fail, user trust is lost.

Although the application supports reminders for:

- Periodic servicing
- Insurance renewal
- Pollution control checks

The automation implemented in this project primarily validates the Oil Change reminder workflow.

OBJECTIVES OF THE PROJECT

The objectives of this project are:

1. To understand mobile automation testing using Appium and Java
2. To design test cases for reminder management
3. To automate Add, Edit, and Delete reminder functionalities
4. To implement screenshot capture on failure
5. To generate automated HTML test reports
6. To simulate real-world mobile testing scenarios

TERMINOLOGIES

1 Appium

Appium is an open-source mobile automation framework used to test Android and iOS applications.

2 Java

Java is used to write automation scripts.

3 Test Automation

Test automation refers to executing test cases using scripts instead of manual testing.

4 Android Application

Mobile application designed for Android operating system.

5 Test Case

A structured scenario to validate a specific functionality.

6 Smoke Testing

Basic validation to ensure major functionality works.

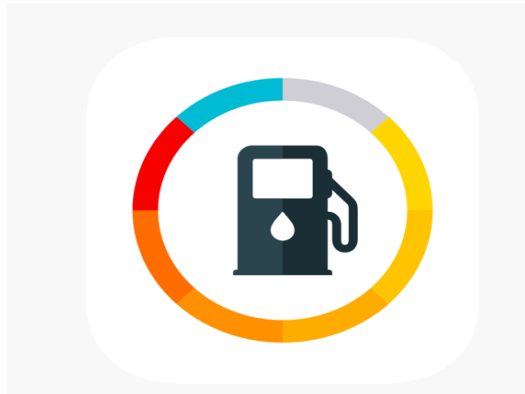
7 Regression Testing

Re-testing application after changes.

8 UI Testing

Validating user interface elements.

APPLICATION OVERVIEW



The application used in this project is **Drivvo**.

Features include:

- Add vehicle
- Track fuel expenses
- Add service reminders
- Maintain service history

This project automates only reminder management

APPLICATION ARCHITECTURE

The mobile application follows layered architecture:

User → UI Layer → Business Logic → Data Storage

When reminder is added:

1. User inputs data
 2. Application validates
 3. Data stored internally
 4. Displayed in reminder list
-

AUTOMATION FRAMEWORK ARCHITECTURE

Automation flow:

Java Script → Appium Client → Appium Server → Android Emulator → Drivvo App

Components:

- UiAutomator2Options
- AndroidDriver
- Locators (ID, XPath)
- Screenshot utility
- HTML report generator

```
Microsoft Windows [Version 10.0.26200.7840]
(c) Microsoft Corporation. All rights reserved.

C:\Users\kurak>appium
[Appium] Welcome to Appium v3.2.0
[Appium] The autodetected Appium home path: C:\Users\kurak\.appium
[Appium] Attempting to load driver uiautomator2...
[Appium] Requiring driver at C:\Users\kurak\.appium\node_modules\appium-uiautomator2-driver\build\lib\index.js
[Appium] AndroidUiautomator2Driver has been successfully loaded in 1.204s
[Appium] Appium REST http interface listener started on http://0.0.0.0:4723
[Appium] You can provide the following URLs in your client code to connect to this server:
http://10.55.23.156:4723/
http://127.0.0.1:4723/ (only accessible from the same host)
[Appium] Available drivers:
[Appium]   - uiautomator2@6.8.0 (automationName 'UiAutomator2')
[Appium] No plugins have been installed. Use the "appium plugin" command to install the one(s) you want to use.
```

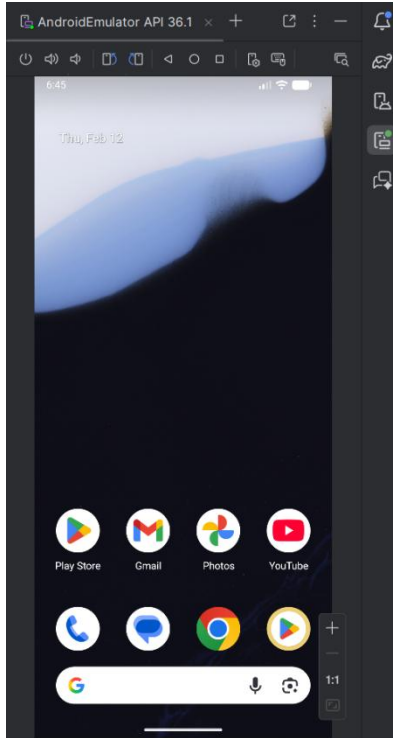
ENVIRONMENT SETUP

Describe:

- Java installation
- Android Studio setup
- Emulator creation

- Appium installation
- Device configuration

Capabilities used in code:



- Platform Name
- Automation Name
- Device Name
- App Package
- App Activity
- No Reset
- New Command Timeout

TEST CASE DESIGN

In this project, test cases were designed to validate the core functionalities of the Car Service & Maintenance Reminder application (Drivvo).

The main focus was to ensure that important features such as adding, editing, and deleting reminders work correctly without any errors.

Before writing automation scripts, the application was explored manually to understand how a user interacts with it. Based on user actions, the following test scenarios were identified.

Key Functionalities Identified

The following core features were selected for automation:

- Add a new service reminder
- Edit an existing reminder
- Delete a reminder

These features are critical because they directly affect the user experience.

Designed Test Cases

TC01 – Add Reminder

Objective:

To verify that a user can successfully add a new service reminder.

Expected Result:

The reminder should be saved and displayed in the reminder list.

TC02 – Edit Reminder

Objective:

To verify that an existing reminder can be modified.

Expected Result:

The updated reminder details should be reflected correctly in the list.

TC03 – Delete Reminder

Objective:

To verify that a reminder can be deleted successfully.

Expected Result:
The reminder should be removed from the list.

DETAILED TEST CASES

TC ID	Test Scenario	Expected Result	Actual Result	Status
TC01	Add Reminder	Reminder saved	Reminder displayed in list	Pass
TC02	Edit Reminder	Updated details saved	Details updated successfully	Pass
TC03	Delete Reminder	Reminder removed	Reminder no longer visible	Pass

The screenshot displays an IDE with the following components:

- Package Explorer:** Shows the project structure with folders like `src/main/java`, `src/main/resources`, `src/test/java`, and `src/test/resources`. The `Demo` folder is expanded, showing `Car_service_reminder.java` and `package-info.java`.
- Code Editor:** Displays the `Car_service_reminder.java` file. The code is a Java class with a `main` method that uses `UiAutomator2Options` to configure an `AndroidDriver` and execute a series of actions: `addReminder()`, `editReminder()`, `deleteReminder()`, and `takeScreenshot()`.
- Outline:** Shows the class structure with methods: `driver - AndroidDriver`, `report - FileWriter`, `main(String[]) - void`, `addReminder() - void`, `editReminder() - void`, `deleteReminder() - void`, and `takeScreenshot(String) - void`.
- Console:** Displays the output of the application, showing the following log messages:

```
<terminated> Car_service_reminder [Java Application] C:\Program Files\Java\jdk-25.0.2\bin\javaw.exe (12 Feb 2026, 7:01:34 pm - 7:03:10 pm elapsed 0:01:35) [pid: 25208]
App Started Successfully
Executing Add Reminder
Add Reminder Passed
Executing Edit Reminder
Edit Reminder Passed
Executing Delete Reminder
Delete Reminder Passed
App Working Properly
```

AUTOMATION IMPLEMENTATION

In this project, automation was implemented using **Appium and Java** to interact with the Drivvo mobile application. The automation framework was designed to simulate real user actions such as clicking buttons, entering text, and validating reminder details.

1. Desired Capabilities:

Desired Capabilities are used to configure the mobile device and application before starting automation.

They provide important information such as:

- Platform name (Android)
- Device name (Emulator or Real Device)
- App package name
- App activity

These details help Appium understand which device and application it needs to control during execution.

In simple terms, DesiredCapabilities act as a bridge between the automation script and the mobile device.

2. Driver Initialization

The Appium driver is responsible for controlling the mobile application.

During driver initialization:

- A connection is established between the Java test script and the Appium server.

- The Appium server then connects to the Android emulator or real device.
- Once the connection is successful, automation can begin.

Without driver initialization, no test actions can be performed.

3. Locator Strategies

Locator strategies are used to identify UI elements in the mobile application.

In this project, the following locators were used:

- ID (Preferred method)
- XPath (Used when ID is not available)

Using stable locators ensures reliable automation and reduces execution failures.

Choosing the correct locator strategy improves test stability and performance.

4. Wait Mechanisms

Mobile applications may take time to load elements.

To handle this, wait mechanisms were used:

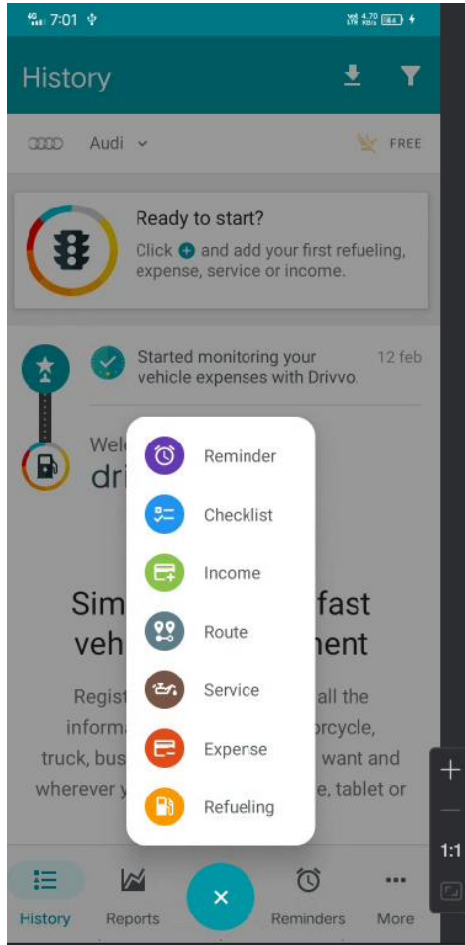
- Implicit Wait
- Explicit Wait

Waits ensure that the script interacts with elements only after they are fully loaded, preventing errors caused by timing issues.

This improves automation reliability.

AUTOMATING ADD REMINDER FUNCTIONALITY

Step 1: Launch the Application



- Appium driver is initialized.
- The Drivvo app is launched on the Android device.
- We wait a few seconds to make sure the app loads properly.

Purpose: Start the application before performing any action.

Step 2: Click on “+” (Add) Icon

- The automation script finds the “+” icon using its ID.
- It performs a click action.

Purpose: This button opens the screen to create a new reminder.

Step 3: Select “Reminder” Option

- After clicking “+”, multiple options appear.
- The script selects the **Reminder** option.

Purpose: We want to create a reminder, not fuel entry or expense.

Step 4: Select “Service”

- The script selects the “Service” category.
- This tells the app that we are adding a service-related reminder.

Purpose: Oil Change comes under service type.

Step 5: Choose “Type of Service”

- The script clicks on “Type of Service”.
- A list of services appears.
- Using Android UI Automator scrolling, the script scrolls down.
- It selects **Oil Change**.

Purpose: Specify what kind of service reminder we are adding.

Step 6: Enter Odometer Value

- The script finds the odometer input field.
- It enters value: **5000** using `sendKeys()`.

Purpose: Reminder should trigger after reaching 5000 km.

Step 7: Enable Date Option

- The script clicks the date checkbox.
- It opens the date picker.
- It selects a specific date (for example: 25 February 2026).
- It clicks OK.

Purpose: Set a reminder date for oil change.

Step 8: Enter Notes

- The script finds the notes field.
- It enters: **“Change engine oil”**

Purpose: Provide additional information for reminder.

Step 9: Click Save Button

- The script clicks on the Save button.
- The reminder gets stored in the application.

VALIDATION STEP

After clicking Save:

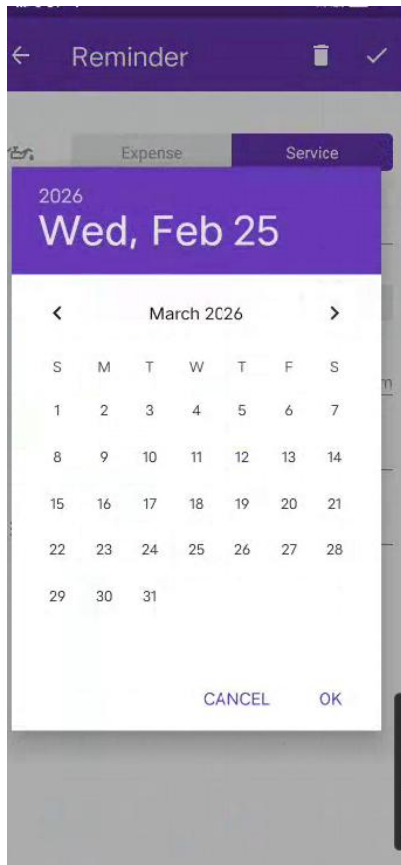
- The script waits for the screen to load.
- It checks whether the Oil Change reminder appears in the reminder list.

If reminder is visible → Test Case PASS

If reminder is not visible → Test Case FAIL (Screenshot captured)

AUTOMATING EDIT FUNCTIONALITY

Step 1: Open Reminder Section



- After the app is launched, the script clicks on the “Reminders” tab at the bottom.

- This opens the list of all saved reminders.

Purpose: We need to access the existing reminder before editing it.

Step 2: Select Existing Reminder

- The script finds the Oil Change reminder from the list.

- It clicks on that reminder.

Purpose: Open the reminder details screen so we can modify it.

Step 3: Clear Odometer Field

- The script locates the odometer input field.
- It uses the `clear()` method to remove the existing value (5000).

Purpose: Remove old value before entering a new one.

Step 4: Enter New Odometer Value

- After clearing, the script enters a new value: **6000** using `sendKeys()`.

Purpose: Update the reminder to trigger at 6000 km instead of 5000 km.

Step 5: Change the Date

- The script clicks on the Date field.
- The date picker opens.
- It clicks on **Next Month**.
- Selects a new date (for example: 10 March 2026).
- Clicks **OK**.

Purpose: Update the reminder schedule date.

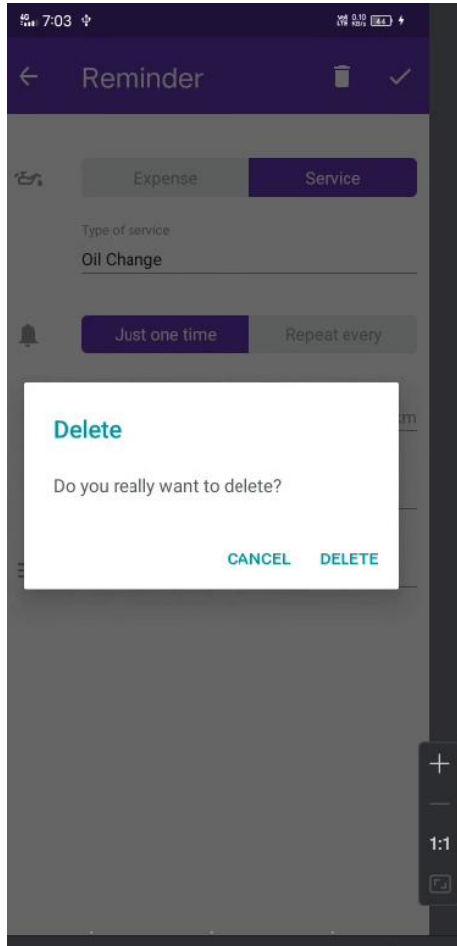
Step 6: Click Save Button

- The script clicks the **Save** button.
- The updated details are stored in the application.

Purpose: Finalize the modification process.

AUTOMATING DELETE REMINDER FUNCTIONALITY

Step 1: Open the Reminder



- After editing is completed, the script stays on the reminder list screen.

- The script finds the Oil Change reminder from the list.

- It clicks on that reminder.

Purpose: Open the reminder details screen before deleting it.

Step 2: Click on Delete Button

- On the reminder details screen, the script locates the **Delete** button.

- It performs a click action on the Delete button.

Purpose: Initiate the deletion process.

Step 3: Confirm Deletion

- A confirmation popup appears asking:
“Are you sure you want to delete?”
- The script clicks on the “**Yes**” button (Confirm option).

Purpose: Confirm that the reminder should be permanently removed.

SYNCHRONIZATION HANDLING

Synchronization is important in mobile automation because the application may take time to load screens or UI elements. If the automation script tries to interact with elements before they are fully loaded, it can result in errors like *element not found*.

In this project, `Thread.sleep()` is used to pause the execution for a few seconds between actions. This ensures that:

- The application screen loads completely
- Buttons and input fields become visible
- The next action is performed only after the previous action finishes

Although `Thread.sleep()` is a simple waiting method and not the most advanced approach, it helps maintain stable test execution in this automation framework.

SCREENSHOT CAPTURE IMPLEMENTATION

Step 1: Detect Test Failure

If any exception occurs during execution, the test is marked as failed.

Step 2: Capture Screenshot

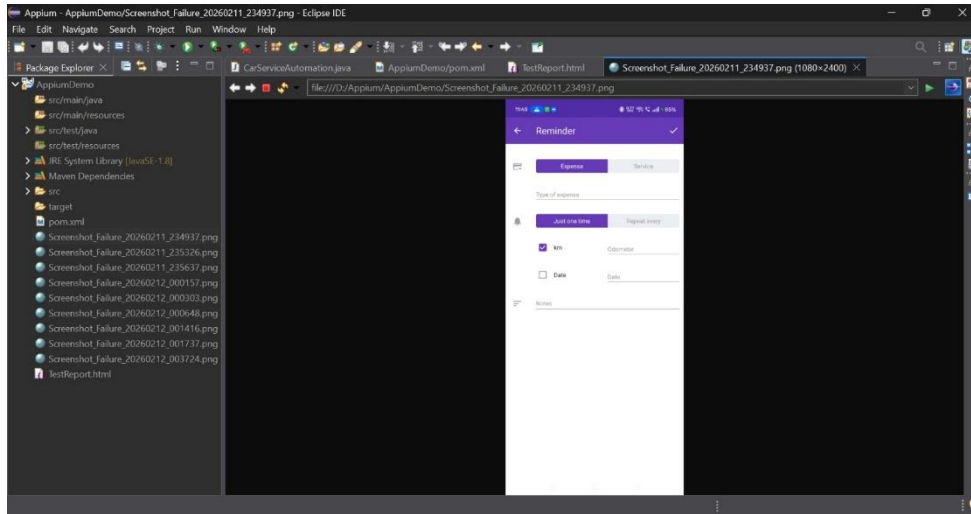
The `getScreenshotAs()` method is used to capture the current screen of the application.

Step 3: Generate Timestamp

A timestamp is created using date and time format to make the screenshot name unique.

Step 4: Save Screenshot

The screenshot file is saved in the project folder with a name like:



TEST EXECUTION PROCESS – Step by Step

Step 1: Start Appium Server

First, the Appium server is started.

This allows communication between your Java automation script and the Android device or emulator.

Without starting Appium, automation cannot run.

Step 2: Connect Emulator or Real Device

Next, the Android emulator (or real device) is launched and connected.

The device must be ready before running the script.

This is where the mobile application will be tested.

Step 3: Run Java Main Class

You run your Java file (`Car_service_reminder.java`) from your IDE (Eclipse/IntelliJ).

This starts the automation execution.

Step 4: Driver Initialization

The **AndroidDriver** is initialized using **UiAutomator2Options**.

This connects:

Java Script → Appium Server → Android Device → Drivvo App

Now the automation session is active.

Step 5: Test Methods Execute

After driver initialization, the following methods run one by one:

- **addReminder()**
- **editReminder()**
- **deleteReminder()**

Each method performs actions and validates the result.

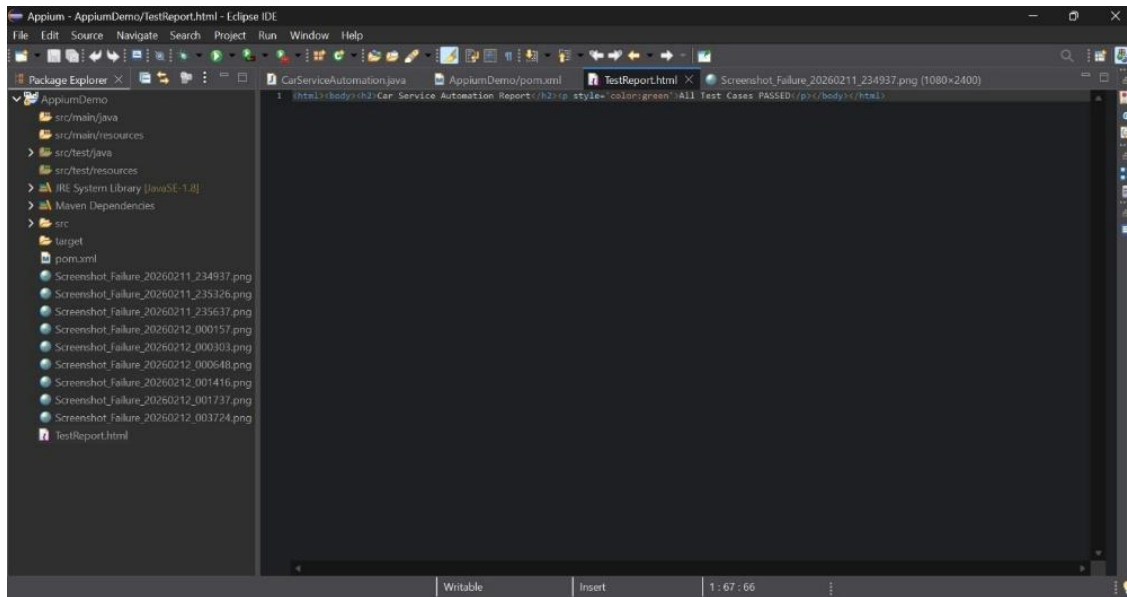
If any error occurs, a screenshot is captured.

Step 6: HTML Report Generated

After all methods complete:

- A simple HTML report (**TestReport.html**) is generated.

- It shows whether the test cases passed or failed.



CHALLENGES FACED

1. Synchronization Issues

Sometimes the app takes time to load screens or buttons.

If the script runs too fast, it tries to click elements before they appear.

This caused “element not found” errors.

2. XPath Stability

Some elements did not have proper ID values.

So XPath was used to locate them.

But XPath can break easily if the UI structure changes, which made automation slightly unstable.

3. Emulator Slowness

The Android emulator sometimes worked slowly. Because it uses system RAM and CPU, the app loading became delayed. This increased execution time and sometimes caused timing issues.

4. Date Picker Handling

Selecting a specific date in the date picker was slightly tricky. The script had to correctly identify the date and click it. If the date format or month changed, it required careful handling.

SOLUTIONS IMPLEMENTED

1. Added Sufficient Wait Time

To solve synchronization issues, proper waiting time was added using `Thread.sleep()`. This helped the app load completely before performing the next action. It reduced “element not found” errors.

2. Used Stable Locators

Whenever possible, ID locators were used because they are faster and more reliable.

XPath was used only when ID was not available.

This improved test stability.

3. Verified Element Selection

Before finalizing the script, each UI element was checked carefully using Appium Inspector.

This ensured that the correct elements were being clicked and updated.

4. Handled Exceptions Properly

A try-catch block was used in the code.

If any test failed, the script captured a screenshot and marked the test as failed.

This made debugging easier and improved reporting.

INDUSTRY RELEVANCE

In real software companies, automation testing is very important. Mobile applications are updated regularly, and every new update must be tested to make sure old features are still working properly.

Automation helps in the following ways:

- It reduces manual testing effort.
- It saves time during regression testing.

- It ensures stable and reliable releases.
- It quickly identifies defects before the app is released to users.

The automation framework developed in this project can be improved further by:

- Using **TestNG** for better test execution and reporting.
- Implementing **Page Object Model (POM)** for better code structure and maintainability.
- Integrating with **CI/CD tools** so tests run automatically whenever new code is added.

Overall, this project reflects how mobile automation is used in real industry environments to improve application quality and reliability.



DELIVERABLES

1. Test Case Document

A document was prepared that contains all the test cases designed for the project.

It includes test case ID, scenario, expected result, actual result, and status (Pass/Fail).

2. Java Automation Script

Automation scripts were written using Java and Appium.

The script includes driver setup, add reminder, edit reminder, delete reminder, screenshot capture, and report generation.

3. HTML Report

After execution, a simple HTML report is generated.

It shows whether the test cases passed or failed.

4. Screenshots

Screenshots are captured automatically if any test case fails.

These screenshots help in identifying and debugging errors.

LEARNING OUTCOMES

Step 1: Understanding Mobile Automation

Learned how mobile applications can be tested automatically instead of manually.

Step 2: Hands-on Experience with Appium

Gained practical experience in using Appium to automate Android applications.

Step 3: Driver Configuration Knowledge

Understood how to initialize AndroidDriver and set desired capabilities properly.

Step 4: Element Identification

Learned how to locate mobile elements using ID, XPath, and AndroidUIAutomator.

Step 5: Handling Synchronization

Understood how to manage timing issues using waits to avoid execution errors.

Step 6: Debugging and Screenshot Capture

Learned how to capture screenshots when test cases fail for easy debugging.

Step 7: Report Generation

Implemented simple HTML report generation to show Pass/Fail results.

FUTURE ENHANCEMENTS – Step by Step

Step 1: Implement Page Object Model (POM)

Organize code better by separating page elements and test logic.

Step 2: Use TestNG Framework

Improve test execution structure and generate advanced reports.

Step 3: Automate Insurance Reminder

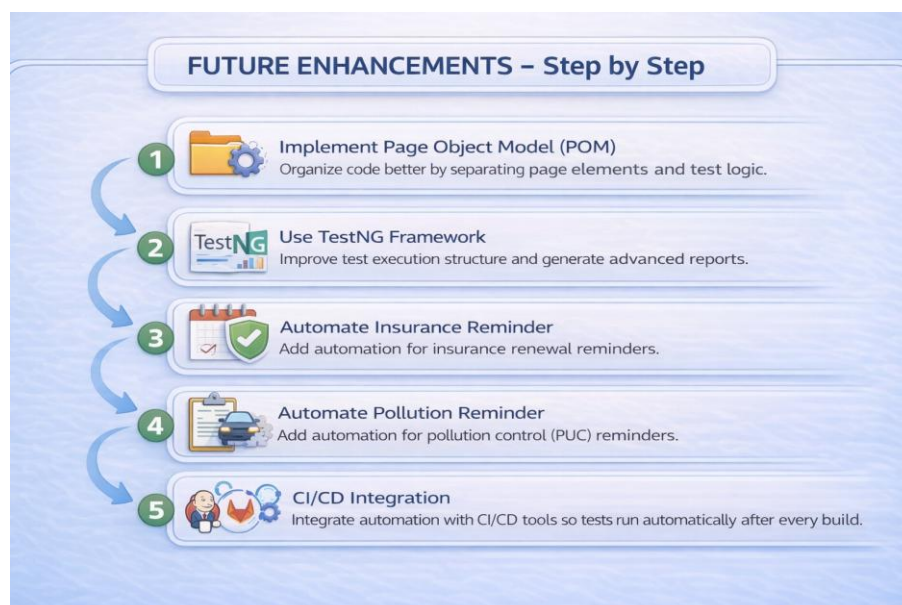
Add automation for insurance renewal reminders.

Step 4: Automate Pollution Reminder

Add automation for pollution control (PUC) reminders.

Step 5: CI/CD Integration

Integrate automation with CI/CD tools so tests run automatically after every build.



CONCLUSION

This project successfully automated the reminder management functionality of a Car Service & Maintenance Reminder mobile application using Appium and Java.

The framework validates:

- Oil Change reminder creation
- Reminder modification
- Reminder deletion

Although the application supports periodic servicing, insurance renewal, and pollution control reminders, this implementation primarily automates the Oil Change workflow.

The project demonstrates real-world mobile automation testing principles and provides a strong foundation for industry-level testing practices.